SECTION 3 · SUBSECTION 3.4
COMPLEXITY SCORING

3.4.1 INTRODUCTION.

Complexity scoring assigns each source file one of four risk levels from independent structural signals: syntax failure, cycle membership, maximum function complexity, average complexity, coupling, undocumented ratio and line count. The one idea the reader must leave with is that the signals combine by OR, not AND. A file reaches the highest level any single signal justifies (figure 3.4-1).

RISK LADDER — HIGHEST REACHED SIGNAL WINS

SIGNAL	MEDIUM AT	HIGH AT	CRITICAL AT
MAX FUNCTION CC	—	11	21
AVERAGE CC	6	10	—
COUPLING (OUT)	4	8	—
UNDOCUMENTED RATIO	0.60	0.80	—
LINE COUNT	300	600	—
PARSE ERROR	—	—	YES
IN CYCLE	—	—	YES

A DASH MEANS THE SIGNAL CANNOT PRODUCE THAT LEVEL ON ITS OWN.
LOW = EVERY SIGNAL BELOW ITS MEDIUM COLUMN; ANY ONE SIGNAL RAISES THE FILE.
UNDOCUMENTED RATIO IS IGNORED BELOW 3 FUNCTIONS PER FILE.

CPX-501A

Figure 3.4-1. Risk Signal Thresholds

3.4.2 FUNCTIONAL DESCRIPTION.

For Python, Radon's *cc_visit* returns per-function complexity; for JavaScript, TypeScript and the generic languages, the scorer walks the syntax tree and counts branch node types (if, loop, catch, ternary, logical operators, switch cases). Files are scored in a thread pool of up to twelve workers. Because coupling comes from the graph, which runs concurrently, the scorer first writes a provisional coupling count from symbol-table imports. *apply_graph_coupling* then replaces it with the graph out-degree, updates cycle membership, and recomputes the verdict for every file.

3.4.2.1 Verdict Rule.

The evaluator checks CRITICAL signals first and returns immediately when one fires. Otherwise it checks each HIGH signal, then each MEDIUM signal. Reasons accumulate, so a file can carry several explanations for one level. A clean file with no function and no signal is LOW with an empty reason list.

```
# src/agents/complexity_scorer.py, _compute_risk (condensed)
if parse_error:           reasons.append("File contains syntax errors"); level = "CRITICAL"
if is_in_circular_dep:    reasons.append("Involved in a circular dependency"); level = "CRITICAL"
if max_complexity >= 21:  reasons.append(f"Function '{_trunc(str(max_complexity))}' ..."); level = "CRITICAL"
if level == "CRITICAL":   return level, reasons        # short-circuit
if max_complexity >= 11:  level = "HIGH"               # then avg, coupling, docs, lines
# note: the CRITICAL reason interpolates the score where the function name belongs
```

3.4.3 PERFORMANCE AND DESIGN DATA.

Table 3.4-1. Risk Thresholds and Fields

Signal	Medium	High	Critical
Max function CC	—	11	21
Average CC	6.0	10.0	—
Coupling (out-degree)	4	8	—
Undocumented ratio	0.60	0.80	—
Line count	300	600	—
Parse error	—	—	yes
In cycle	—	—	yes
Undoc. gate (functions)	≥3	≥3	—

CAUTION

These are heuristics, not defects. A generated parser with a 25-branch switch is CRITICAL by rule even if it is correct. Use the level to choose what to read first, not as a quality judgement. The recorded reason text for CRITICAL max-CC is malformed; see 8.1-1.

SCORE CHECK	MAINTAINER
1 PICK - a small clean file	
2 CONFIRM - LOW, no reasons	
√ CC - matches manual count	
3 CHECK - coupling equals graph out-degree	
4 VERIFY - cycle file is CRITICAL	

3.4.4 WORKED READING.

A 720-line file with average complexity 4.0 and a worst function at 23 is CRITICAL, not HIGH: the evaluator returns on the critical signal before it tests line count. A worst function at 12 in the same file is HIGH twice over — the complexity and line-count reasons are both kept.